Composite

Composite · Compute total size of nested project folders · Build with Waqar

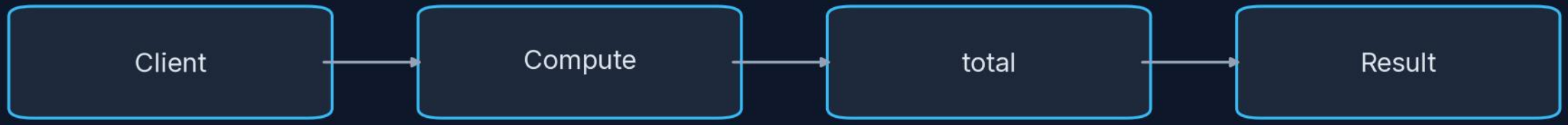
Today's Hook

- Today's force: Compute total size of nested project folders
- We earn Composite by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: FsNode, FileNode, DirNode

The Problem

- Scenario: Compute total size of nested project folders
- Constraints: solve «Compute total size of nested project folders» with Composite — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Compute total size of nested project folders
- Success looks like: 30

Structure



Composite: Compute total size of nested project folders

Real-World Scenario

- Running example: Compute total size of nested project folders
- Key types: FsNode, FileNode, DirNode
- Happy-path stdout cue: 30
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

public final class NaiveSize {
  public static int total(Map<String,Integer> flat){ return flat.values().stream().mapToInt(i->i).sum(); }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Compute total size of nested project folders
- Without Composite, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

import java.util.*;
sealed interface FsNode permits FileNode, DirNode { String name(); int size(); }
record FileNode(String name, int size) implements FsNode {}
final class DirNode implements FsNode {
  private final String name; private final List<FsNode> kids = new ArrayList<>();
  public DirNode(String name){ this.name=name; }
  public DirNode add(FsNode n){ kids.add(n); return this; }
  public String name(){ return name; }
  public int size(){ return kids.stream().mapToInt(FsNode::size).sum(); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

public final class ProblemDemo {
  public static void main(String[] args) {
    System.out.println("=== Composite: nested folder sizes ===");
    System.out.println("--- naive flat map ---");
System.out.println(NaiveSize.total(Map.of("a.txt",10,"b.txt",20)));
System.out.println("--- composite ---");
var root = new DirNode("src").add(new FileNode("A.java",10))
  .add(new DirNode("util").add(new FileNode("B.java",25)));
System.out.println("size="+root.size());
  }
}
```

Walkthrough Output

```
=== Composite: nested folder sizes ===
--- naive flat map ---
30
--- composite ---
size=35
```

Step-by-Step Trace

- Match stdout to stages — look for: 30
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 10-composite ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.gof.structural.composite.problem;

import java.util.*;

import java.util.*;
sealed interface FsNode permits FileNode, DirNode { String name(); int size(); }
record FileNode(String name, int size) implements FsNode {}
final class DirNode implements FsNode {
  private final String name; private final List<FsNode> kids = new ArrayList<>();
  public DirNode(String name){ this.name=name; }
  public DirNode add(FsNode n){ kids.add(n); return this; }
  public String name(){ return name; }
  public int size(){ return kids.stream().mapToInt(FsNode::size).sum(); }
}
```

Trade-offs

- Use Composite when «Compute total size of nested project folders» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Composite in product language.

Related Patterns

- Name the pattern usually confused with Composite and separate the forces.
- Sketch the roles for «Compute total size of nested project folders» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Compute total size of nested project folders
- Naive pain → Composite boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.